North Bengal International University

Lab Report: 03

Exp Name : Insertion sort & Selection sort Algorithm

Subject : Analysis of algorithm Sessional

Subject Code : CSE 0612-2212

Submitted By:

Name : Md. Tanvir Rahman Pranto
Dept : CSE(26)
Roll : 0712340012101005
Semester : 3rd Year, 5th semester.

Submitted To:

Name : Afsana Tasnim
Designation : (HOD)Lecturer
Dept : CSE
Date :

Task No: 01

Task Name: Insertion sort

Introduction: Insertion Sort is a **simple sorting algorithm** that works the same way people arrange playing cards in their hands. It builds the final sorted array one element at a time by picking each element and placing it in its correct position relative to the already sorted part. It is efficient for **small datasets** or when the list is **already partially sorted**, but not suitable for very large datasets compared to advanced algorithms like Quick Sort or Merge Sort.

Algorithm:

- Start with the second element (index 1) because a single element (index 0) is already sorted.
- Take the current element (key) and compare it with elements in the sorted part (on the left).
- Shift all elements that are greater than the key to one position right.
- Insert the key at its correct position.
- Repeat this for all elements until the array is fully sorted.

Pseudocode:

```
InsertionSort(A, n):
  for i = 1 to n-1 do
    key = A[i]
    j = i - 1

    while j >= 0 and A[j] > key do
      A[j + 1] = A[j]
      j = j - 1
    end while

    A[j + 1] = key
  end for
```

code:

```c
#include <stdio.h>

int main() {
    int n, i, j, key;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("\nArray before sorting:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Insertion Sort
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }


    printf("\nArray after sorting:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

output:

```
Enter the number of elements: 8
Enter 8 elements:
12 4 8 3 6 0 4 1

Array before sorting:
12 4 8 3 6 0 4 1

Array after sorting:
0 1 3 4 4 6 8 12
```

Conclusion: Insertion Sort is an easy-to-understand algorithm that works well for small or nearly sorted datasets. It sorts elements by inserting each element into its correct place among already sorted elements. Although simple, it is inefficient for large datasets due to its $O(n^2)$ time complexity.

Task No: 02

Task Name: Selection sort

Introduction: Selection Sort is a **simple comparison-based sorting algorithm**. It works by **dividing the array into two parts**: a sorted part (on the left) and an unsorted part (on the right). In each step, the smallest (or largest, depending on order) element from the unsorted part is selected and swapped with the first unsorted element. This process continues until the entire array is sorted.

Algorithm:

- Start with the first element (index 0).
- Find the **smallest element** in the unsorted part of the array.
- Swap this smallest element with the first element of the unsorted part.
- Move the boundary between sorted and unsorted part one step forward.
- Repeat the process until the whole array is sorted.

Pseudocode:

```
SelectionSort(A, n):
  for i = 0 to n-2 do
    minIndex = i

    for j = i+1 to n-1 do
```

```
        if A[j] < A[minIndex] then
            minIndex = j
        end if
    end for

    if minIndex != i then
        swap A[i] and A[minIndex]
    end if
  end for
```

code:

```c
#include <stdio.h>
int main() {
    int n, i, j, minIndex, temp;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("\nArray before sorting:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Selection Sort
    for (i = 0; i < n - 1; i++) {
        minIndex = i;

        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        // Swap if a smaller element is found
        if (minIndex != i) {
            temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }
```

```
    printf("\nArray after sorting:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

Output:

```
Enter the number of elements: 5
Enter 5 elements:
4 5 6 9 0

Array before sorting:
4 5 6 9 0

Array after sorting:
0 4 5 6 9
```

Conclusion: Selection Sort is another simple sorting algorithm that works by repeatedly finding the smallest element from the unsorted part and placing it in its correct position. It always makes the same number of comparisons, even if the array is partially sorted. Like Insertion Sort, it has a time complexity of $O(n^2)$ and is not suitable for large datasets, but useful for learning and small inputs.